
```
>>> a = 256
>>> b = 256
❶ >>> a is b
True
>>> c = 257
>>> d = 257
❷ >>> c is d
False
```

All 256 objects are really the same object, so the `is` operator for `a` and `b` returns `True` ❶. But Python created separate 257 objects for `c` and `d`, which is why the `is` operator returns `False` ❷.

The expression `257 is 257` evaluates to `True`, but CPython reuses the integer object made for identical literals in the same statement:

```
>>> 257 is 257
True
```

Of course, real-world programs usually only use an integer's value, not its identity. They would never use the `is` operator to compare integers, floats, strings, booleans, or values of other simple data types. One exception occurs when you use `is None` instead of `== None`, as explained in “Use `is` to Compare with `None` Instead of `==`” on page 96. Otherwise, you'll rarely run into this problem.

String Interning

Similarly, Python reuses objects to represent identical string literals in your code rather than making separate copies of the same string. To see this in practice, enter the following into the interactive shell:

```
>>> spam = 'cat'
>>> eggs = 'cat'
>>> spam is eggs
True
>>> id(spam), id(eggs)
(1285806577904, 1285806577904)
```
